



SECURITY DESIGN

Skills Accelerator Cloud Platform

DOCUMENT	As-built design and security gap assessment
STATUS	Demonstration environment - production hardening required
AWS REGION	af-south-1
ASSESSMENT DATE	22 August 2026

Repository-based assessment covering AWS infrastructure, application security, secrets, logging, IAM, CI/CD, and Terraform state.

Security posture at a glance

Overall classification: **demonstration environment**. The design has meaningful infrastructure controls, but critical application and operational risks must be closed before production use.

IMPLEMENTED STRENGTHS	IMMEDIATE PRIORITIES
- Private EC2 and RDS tiers with security-group chaining - EC2 instance role and parameter-specific secret retrieval - Encrypted RDS storage and centralized EC2/application logs - No inbound SSH rule; Session Manager is the intended admin path	- Authenticate and authorize the public CRUD API - Remove the stored-XSS path and add input/output controls - Add HTTPS and verify the RDS TLS certificate - Recover and migrate Terraform state to a protected remote backend - Replace long-lived operator and SSH credentials with temporary access

Contents

1. Purpose and scope

2. Security objectives

3. System and trust-boundary overview

4. Assets and data classification

5. Implemented security controls

6. Secret and request flows

7. Threat model

8. Security gaps and remediation plan

9. Security verification checklist

10. Operational security procedures

11. Residual-risk statement

12. Reference material

1. Purpose and scope

This document describes the security architecture of the Skills Accelerator Cloud Platform, the controls currently implemented in code, the main threats to the system, and the changes required before the platform should be treated as production-ready.

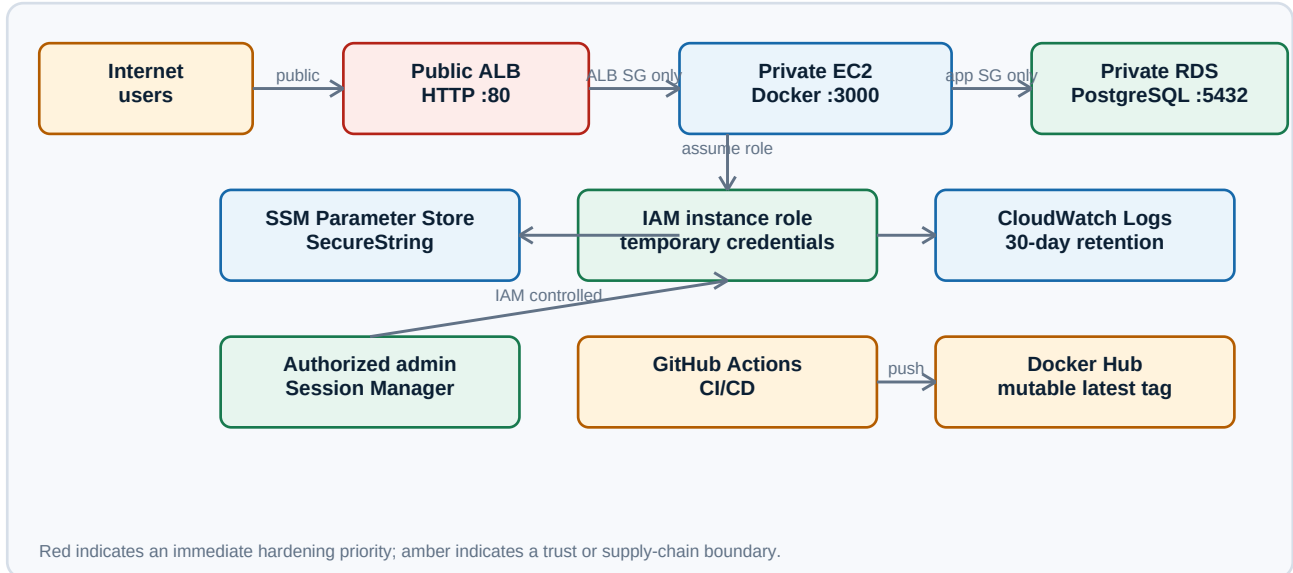
The assessment is based on the repository configuration. It is not a live AWS compliance assessment. The original complete Terraform state is currently unavailable on the home PC, so deployed resources and runtime settings must be reconfirmed after the original state is recovered or the environment is inventoried.

2. Security objectives

The design aims to provide:

- **Confidentiality:** protect the database password, application data, AWS credentials, logs, and Terraform state.
- **Integrity:** prevent unauthorized modification of application records, infrastructure, images, and deployment workflows.
- **Availability:** distribute application instances across Availability Zones and replace unhealthy instances automatically.
- **Auditability:** centralize operating-system audit, bootstrap, and application logs.
- **Least privilege:** give each network tier and workload only the access it needs.

3. System and trust-boundary overview



Primary trust boundaries

1. **Internet to ALB:** untrusted public requests enter the platform.
2. **ALB to application tier:** traffic is limited by security-group identity and port.
3. **Application to database tier:** database access is restricted to application instances.
4. **EC2 to AWS control-plane services:** the instance role authorizes SSM, Parameter Store, and CloudWatch operations.

5. **CI/CD to runtime:** external source code, GitHub Actions, Docker Hub, and the EC2 deployment path form a software-supply-chain boundary.
6. **Terraform operator to AWS:** local credentials and Terraform state can control or expose the complete environment.

4. Assets and data classification

Asset	Classification	Main protection requirement
RDS administrator password	Secret	Never commit, log, or expose publicly; restrict retrieval and rotate
AWS access keys and GitHub deployment secrets	Secret	Prefer temporary credentials; protect with MFA and least privilege
Terraform state	Highly sensitive	Encrypt, restrict access, version, lock, and back up
Skills application records	Internal application data	Authorize writes, validate input, encrypt in transit and at rest
EC2 role credentials	Temporary secret	Require IMDSv2 and limit role permissions
Audit, bootstrap, and application logs	Sensitive operational data	Restrict readers, define retention, detect tampering and suspicious activity
Container image and workflow	Trusted executable content	Pin, scan, sign, and deploy through an authenticated path

5. Implemented security controls

5.1 Network segmentation

- The internet-facing ALB is deployed across two public subnets.
- Auto Scaling instances run in two private subnets and receive application traffic only on TCP port 3000 from the ALB security group.
- RDS runs in a private DB subnet group and is configured with `publicly_accessible = false`.
- PostgreSQL ingress on TCP port 5432 is permitted only from the application security group.
- No SSH ingress rule is defined for the application instances.
- Private instances use the NAT Gateway for outbound software downloads and access to public AWS service endpoints.

This security-group chaining is stronger than allowing application and database ports from broad CIDR ranges. Administrative access is intended to use Systems Manager Session Manager, which avoids opening inbound SSH ports or managing host keys ([AWS Session Manager](#)).

5.2 Workload identity and IAM

- EC2 instances receive temporary AWS credentials through the `skills-accelerator-ec2-ssm-role` instance profile; no AWS access key is embedded in the application or launch template.
- `AmazonSSMManagedInstanceCore` supports Systems Manager management.
- The database secret policy allows only `ssm:GetParameter` and scopes the resource to the single `/skills-accelerator/database/password` parameter ARN.
- `CloudWatchAgentServerPolicy` allows the instance to publish logs.

- The GitHub workflow declares `permissions: contents: read`, which limits the default GitHub token at the repository level.

Using roles for EC2 workloads follows AWS guidance to use temporary workload credentials instead of distributing long-lived access keys ([AWS IAM security best practices](#)).

5.3 Secret handling

- The RDS password variable is marked `sensitive` in Terraform.
- The password is stored in Parameter Store as a `SecureString`.
- EC2 retrieves the value at boot with `--with-decryption` instead of placing the Terraform variable directly in launch-template user data.
- Retrieval retries for up to two minutes and fails closed if no password is returned.
- `.env`, `*.tfvars`, Terraform state, private keys, and Terraform plan files are excluded from Git.

Parameter Store uses AWS KMS to encrypt `SecureString` values. Because no customer-managed key is configured, this design uses the account's AWS-managed `aws/ssm` key ([AWS Parameter Store encryption](#)).

5.4 Data protection

- RDS storage encryption is enabled with `storage_encrypted = true`. RDS encryption covers the underlying storage and associated logs, backups, and snapshots ([AWS RDS encryption](#)).
- The Node PostgreSQL client enables TLS when the database host is an RDS endpoint.
- Database queries use positional parameters rather than concatenating user values into SQL, reducing SQL-injection exposure.
- CloudWatch Logs encrypts log data at rest using its default server-side encryption; a customer-managed KMS key is not currently associated with the log group ([CloudWatch Logs encryption](#)).

5.5 Logging and monitoring

- The `/skills-accelerator/ec2/audit` CloudWatch log group has a 30-day retention period.
- The CloudWatch Agent forwards `/var/log/audit/audit.log` and `/var/log/cloud-init-output.log` into per-instance streams.
- Docker uses the `awslogs` logging driver and writes application logs to a per-instance application stream.
- Linux `auditd` is installed and enabled during bootstrap.
- CPU alarms invoke scale-up at an average CPU utilization above 70% and scale-down below 30%.
- Auto Scaling uses ELB health checks to replace unhealthy application instances.

The CPU alarms are availability controls, not security alerts. No SNS notification, CloudTrail trail, VPC Flow Logs, ALB access logging, GuardDuty, AWS Config, or WAF resource is defined in the current Terraform.

6. Secret and request flows

6.1 Database-secret flow

1. Terraform receives `db_password` as a sensitive input.
2. Terraform configures the RDS administrator password and writes the same value to an SSM `SecureString` parameter.
3. At boot, the EC2 instance assumes its IAM role.

4. The boot script calls `GetParameter` for the single approved parameter with decryption enabled.
5. The password is passed into the Docker container as `DB_PASSWORD`.
6. The application opens a TLS connection to RDS.

The secret is no longer embedded directly in EC2 user data. However, it remains present in Terraform state because Terraform manages both the RDS password and the Parameter Store value. It is also present in the running container's environment and can be read by a principal with Docker or root-level access.

6.2 Application-request flow

1. A public client connects to the ALB on HTTP port 80.
2. The ALB forwards the request over HTTP to the target group on port 3000.
3. The application performs unauthenticated CRUD operations against PostgreSQL.
4. PostgreSQL is reached only through the RDS security-group rule from the application tier.
5. Application output is written to CloudWatch Logs through the Docker logging driver.

7. Threat model

Threat actor or event	Security concern	Existing control	Remaining exposure
Anonymous internet user	Unauthorized record creation, modification, or deletion	ALB and private tiers	API has no authentication or authorization
Malicious application input	Stored cross-site scripting or malformed data	Parameterized SQL	Frontend inserts database values into <code>innerHTML</code> ; server lacks schema validation
Network observer	Credential or data interception	RDS connection uses TLS	Public ALB traffic is HTTP; RDS certificate verification is disabled
Compromised EC2 process	Theft of role credentials or DB password	IAM role and parameter-specific <code>GetParameter</code>	IMDSv2 is used by the script but not enforced in the launch template; secret is a container environment variable
Compromised operator credentials	Full infrastructure modification or data access	Separate IAM user	Long-lived access keys, broad user policies, and missing MFA were observed during project setup
Compromised dependency or CI action	Malicious application deployment	GitHub secrets and read-only repository token	Mutable image tags, unpinned base image/action, no image signing or vulnerability gate
State-file loss or disclosure	Terraform loses ownership mapping or exposes secrets	State is Git-ignored; a partial local backup exists	No remote backend, centralized backup, encryption policy, or locking
Accidental deletion	Permanent database loss	RDS storage encryption	<code>skip_final_snapshot = true</code> ; deletion protection and explicit backup policy are absent

8. Security gaps and remediation plan

The following priorities distinguish the current learning/demo environment from a production-ready deployment.

Priority	Finding	Required remediation
P0 - Critical	All create, update, and delete API routes are publicly accessible without authentication or authorization.	Add user authentication and server-side authorization. Restrict mutation operations to authenticated, permitted identities.
P0 - Critical	User-controlled database content is rendered through <code>innerHTML</code> , creating a stored-XSS path.	Render untrusted values with <code>textContent</code> or a proven output-encoding/sanitization library. Add a restrictive Content Security Policy.
P0 - High	The ALB accepts only HTTP port 80.	Request an ACM certificate, add an HTTPS 443 listener, select a current TLS policy, and redirect HTTP to HTTPS. AWS requires a certificate for an HTTPS ALB listener (AWS ALB HTTPS certificates).
P0 - High	Terraform state is local, incomplete on the home PC, and contains sensitive values.	Recover or reconstruct the full state, then migrate it to a protected remote backend. For S3, enable bucket versioning, encryption, restricted IAM, public-access blocking, and <code>use_lockfile = true</code> (Terraform S3 backend). Never commit state to Git.
P0 - High	The current operator model uses a long-lived IAM-user access key; MFA was not enabled when reviewed.	Enable MFA, remove unnecessary broad policies, rotate and remove unused keys, avoid root-user administration, and move to IAM Identity Center or another temporary-credential workflow.
P1 - High	The PostgreSQL client sets <code>rejectUnauthorized: false</code> .	Install the current Amazon RDS CA bundle and enable full certificate and hostname verification. AWS documents <code>verify-full</code> for protection against spoofing (RDS PostgreSQL TLS).
P1 - High	RDS final snapshots, deletion protection, Multi-AZ, and an explicit backup-retention policy are not configured.	Enable deletion protection for retained environments, define automated backups, take a final snapshot before destructive changes, and evaluate Multi-AZ based on recovery requirements.
P1 - High	The CI/CD workflow deploys with an SSH private key to one host, while the Terraform design uses private, Auto Scaling-managed instances with no SSH ingress.	Replace host-based SSH deployment with GitHub OIDC to a narrowly scoped AWS role and an Auto Scaling instance-refresh or SSM-based deployment. Do not copy container environment variables into <code>/tmp</code> .
P1 - Medium	The launch template requests an IMDSv2 token but does not enforce IMDSv2.	Add launch-template metadata options with the endpoint enabled, HTTP tokens required, and an appropriate hop limit for the container design (AWS IMDSv2 configuration).
P1 - Medium	<code>CloudWatchAgentServerPolicy</code> and outbound security-group rules are broader than the application's exact needs.	Replace the logging policy with actions scoped to the project log group where AWS supports resource scoping. Restrict egress and consider VPC endpoints for SSM, EC2 Messages, SSM Messages, CloudWatch Logs, and ECR if the image registry is migrated.
P1 - Medium	The secret uses the default <code>aws/ssm</code> key.	Use a customer-managed KMS key and grant <code>kms:Decrypt</code> only to the intended role and parameter encryption context. AWS recommends a customer-managed key for finer access control (Systems Manager security practices).
P2 - Medium	The Docker image uses <code>latest</code> ; the base image and third-party GitHub Action are not pinned to immutable digests/commits. The container runs as root.	Pin image digests and action commit SHAs, use <code>npm ci</code> , scan dependencies/images, produce an SBOM, sign the image, verify it before deployment, and run the application as a non-root user.
P2 - Medium	No WAF, ALB access logs, CloudTrail trail, VPC Flow Logs, GuardDuty, Config, or security notifications are defined.	Add controls appropriate to the project's risk and budget. Alert on IAM changes, denied secret access, unusual sessions, application errors, and destructive database activity.
P2 - Medium	Application input has no explicit schema, length, type, or allowed-value validation and no rate limiting.	Validate every request server-side, reject unexpected fields, enforce size limits, add security headers, and rate-limit public routes.

9. Security verification checklist

Before describing the environment as production-ready, record evidence for each item:

- ☐ Public HTTP redirects to HTTPS, and TLS configuration passes an external test.
- ☐ Anonymous users cannot create, update, or delete records.
- ☐ Stored-XSS test strings render as text and never execute.
- ☐ RDS certificate and hostname validation succeeds using the current AWS CA bundle.
- ☐ RDS is private, encrypted, backed up, and protected from accidental deletion.
- ☐ EC2 has no public IP and no inbound SSH rule.
- ☐ IMDSv1 requests fail and IMDSv2 requests succeed.
- ☐ The EC2 role can retrieve only the intended parameter and cannot retrieve other secrets.
- ☐ Parameter and log access is recorded and reviewed.
- ☐ CloudWatch receives audit, cloud-init, and application streams from every current instance.
- ☐ The deployment uses temporary AWS credentials and no long-lived SSH key.
- ☐ The deployed container image matches an approved immutable digest and scan result.
- ☐ Terraform state is remotely stored, encrypted, versioned, locked, backed up, and accessible only to approved operators.
- ☐ Root and human administrative access uses MFA; unused access keys and excessive policies are removed.
- ☐ A reviewed `terraform plan -destroy` is required before environment teardown, and the required final database snapshot exists.

10. Operational security procedures

Credential handling

- Never paste access keys, secret keys, database passwords, private SSH keys, or state contents into chat, screenshots, issues, or commits.
- Use named local AWS profiles only when temporary federation is unavailable.
- Rotate a credential immediately if it is displayed or shared outside its intended secret store.
- Review access-key last-used information and remove unused keys.

Change management

- Require pull-request review for Terraform, application, Dockerfile, and workflow changes.
- Run formatting, validation, security scanning, dependency checks, and a reviewed Terraform plan before deployment.
- Do not apply or destroy infrastructure from a partial or unknown state.
- Retain an auditable record of the approved plan and deployed image digest.

Incident response

If an AWS or application credential is suspected of compromise:

1. Disable or revoke the credential without waiting for code changes.
2. Preserve CloudTrail, Session Manager, CloudWatch, ALB, and application evidence.
3. Identify affected resources, sessions, secret reads, and deployments.
4. Rotate the database password and update the approved secret store.
5. Replace affected instances and containers from a trusted immutable image.
6. Correct the access path, verify controls, and document the incident before restoring service.

11. Residual-risk statement

The current repository demonstrates meaningful defense-in-depth for a learning project: private compute and database tiers, security-group isolation, instance roles, a resource-specific Parameter Store permission, encrypted RDS storage, Session Manager support, and centralized EC2 logging. However, the public HTTP endpoint, unauthenticated write API, stored-XSS path, unverified database certificate, local Terraform state, long-lived operator credentials, and mutable deployment chain mean the system should currently be classified as a **demonstration environment, not a production-ready service**.

Production approval should be withheld until all P0 findings are closed and P1 findings have either been remediated or explicitly accepted by the system owner.

12. Reference material

- [AWS IAM security best practices](#)
- [AWS Systems Manager SecureString encryption](#)
- [AWS ALB HTTPS certificates](#)
- [AWS RDS for PostgreSQL TLS](#)
- [CloudWatch Logs retention](#)
- [Terraform S3 backend](#)
- [AWS Systems Manager Session Manager](#)
- [AWS EC2 IMDSv2 configuration](#)
- [AWS RDS encryption](#)
- [CloudWatch Logs encryption](#)
- [Terraform state and sensitive data](#)